Distributed Computing Engine: A Resilient Mini-Framework for Cryptographic Forensics

Project Proposal · Parallel & Distributed Computing · Instructor: Dr. Mohammed Alaa Ala'anzy

Group: Atabek Bakhytzhanov (240107104) · Begzhan Imangali (240107042) · Karzhaubay Bakdaulet (240107105) · Yerkebulan Kenzhegali (240107051) · Madiyar Yerlanuly (240107040)

1. Motivation and Research Question

In distributed computing, most projects merely write a static program that utilizes MPI libraries. However, production environments (like Slurm or Apache Spark) require an underlying **fault-tolerant framework** that actively schedules tasks, mitigates crashes, and maximizes local CPU performance [4]. We propose building *Mini-Spark*: a custom C-based distributed execution engine leveraging OpenMP SIMD instructions and MPI asynchronous communication. To benchmark the engine, we will deploy a computationally bound demo application: a distributed SHA-256 cryptographic brute-forcer for digital investigations [3]. **We ask: How effectively can a custom distributed framework isolate network logic (asynchronous MPI) from compute logic (OpenMP SIMD), and what is the exact recovery overhead when utilizing replication-based fault tolerance compared to crash-intolerant MPI implementations [1, 2]?**

2. Hypotheses

H1 (Hardware-Agnostic CPU Acceleration): By decoupling the compute engine and explicitly utilizing advanced OpenMP SIMD directives (`#pragma omp simd`) instead of relying on GPUs, the framework will achieve near 95% CPU utilization on heterogeneous x86 architectures, drastically outperforming scalar multi-threading [5].
H2 (Failure Masking via Checkpointing): If a worker node goes offline (simulated network failure), the fault-tolerance daemon will detect the lost heartbeat within 500ms. By maintaining distributed checkpoints, the scheduler will reassign the lost chunk with a maximum performance penalty of 15% during the recovery epoch.
H3 (Dynamic Load Balancing): An adaptive scheduler will maintain >90% weak-scaling efficiency across 4 asymmetric worker nodes by dynamically shrinking task chunks for slower nodes, outperforming static MPI distributions.

3. Methodology

Architecture & Implementation (C/C++): We completely abandon CUDA and build a 5-tier architecture purely for CPU clusters. **(i) The Scheduler:** Manages a dynamic task queue using chunk-based load balancing. **(ii) Network Engine:** Uses strictly non-blocking MPI collectives (`MPI_Isend`, `MPI_Irecv`) to avoid deadlocks. **(iii) Fault Tolerance:** A dedicated daemon that listens for node heartbeats and manages state checkpoints [2]. **(iv) OpenMP Engine:** Local worker daemon that utilizes OpenMP threads and AVX2 vectorization to compute the payload. **(v) Demo Application:** A forensic password cracker that feeds hashes into the framework [3]. **Validation:** We will run the cluster on 5 distinct laptops, actively disconnecting computers from the Wi-Fi network mid-execution to prove the resilience algorithms.

4. Team Roles and Timeline

S1 (Scheduler): Dynamic load balancer and master task queue · **S2 (Network):** Asynchronous MPI engine and heartbeats · **S3 (Rescuer):** Checkpointing and Fault-tolerance state recovery · **S4 (Accelerator):** Advanced OpenMP/SIMD execution engine · **S5 (Data Sci):** Cryptography payload integration and scaling analytics.

W6-W7	Architecture design; Serial cryptography logic; Basic network skeleton.	**W11**	Fault Tolerance (H2): Heartbeats and Checkpoint recovery implementation.
W8	OpenMP SIMD engine (H1); Asynchronous MPI messaging design.	**W12**	Buffer (Exam 2); Integration of all 5 physical laptops via WLAN MPI.
W9	Dynamic Scheduler implementation (H3); chunk size tuning.	**W13**	Chaos testing: physically disconnecting nodes to trigger the Rescuer engine.
W10	**Progress report**: CPU/SIMD scaling baseline (without failures).	**W14-W15**	Final report submission, presentation, and live 5-laptop cluster demo.

References (Strictly 2024–2026)

[1] N. Ali *et al.*, "FTHP-MPI: Towards Providing Replication-based Fault Tolerance in a Fault-Intolerant Native MPI Library," arXiv:2504.09989, 2025.
[2] J. Edwards *et al.*, "The Case for ABI Interoperability in a Fault Tolerant MPI," arXiv:2503.11138, 2025.
[3] K. Wang *et al.*, "Optimizing Password Cracking for Digital Investigations," arXiv:2504.03347, 2025.
[4] M. Rossi *et al.*, "Towards an Adaptive Runtime System for Cloud-Native HPC," arXiv:2603.14630, 2026.
[5] S. Johnson *et al.*, "Parallelizing the Factorial Space: Multi-Core OpenMP Scaling and Scalable SIMD Acceleration," arXiv:2609.07862, 2026.